



DOCUMENTATION TECHNIQUE

Modèle de contenu

Référence des collections, réglages et règles d'accès du CMS

Référence technique du contenu géré dans Payload CMS. Le guide destiné au propriétaire est `GUIDE_ADMIN.md` (Guide_administrateur.pdf).

- Fichiers : `src/payload/collections/`, `src/payload/globals/`, `src/payload/fields/shared.ts`, `src/payload.config.ts`.
- Types générés : `src/payload-types.ts` (`npm run generate:types`).
- Accès public : les pages du site lisent le CMS via `src/lib/cms.ts`, qui convertit les documents en modèles de vue typés (`src/lib/types.ts`) et retombe sur le contenu de démarrage si la base est indisponible.

Sommaire

Conventions communes

Collections

expertise-areas — Domaines d'expertise

experiences — Expériences et projets

subscribers — Abonnés newsletter

insights — Articles RK Insights

categories — Catégories éditoriales

books — Livres et publications

orders — Commandes (vente directe)

products — Produits numériques

protected-files — Fichiers protégés (upload)

members — Membres

entitlements — Accès aux produits

campaigns — Pages de campagne

businesses — Écosystème entrepreneurial

engagements — Conférences & médias

credentials — Formations et certifications

legal-pages — Pages légales

media — Médiathèque (upload)

documents — Documents PDF (upload)

contact-submissions — Demandes de contact

users — Comptes

Globals

site-settings

appearance — Apparence (Administrateur uniquement)

shop-settings — Réglages de la boutique

home-page

about-page

Règles d'accès

Contenu de démarrage et repli

Faire évoluer le modèle

Conventions communes

Champ	Présent sur	Rôle
<code>slug</code>	tous les contenus adressables	Localisé : une URL par langue. Généré depuis le titre s'il est laissé vide
<code>seo</code> (groupe)	contenus + pages	<code>title</code> , <code>description</code> , <code>image</code> , <code>noindex</code> — localisés
<code>isPlaceholder</code>	contenus	Marque un contenu d'exemple : bandeau public + rappel dans l'admin
<code>order</code>	listes ordonnées	Tri croissant
<code>_status</code>	collections versionnées	<code>draft</code> / <code>published</code>

Aperçu des brouillons : le bouton *Aperçu* (`admin.preview`, fonction commune `previewUrl` de `src/payload/preview.ts`) existe sur `insights`, `books`, `products`, `expertise-areas`, `experiences`, `engagements`, `campaigns`, `legal-pages` (page de détail) et `businesses` (page Entreprises). Il ouvre `/api/preview`, qui active le mode brouillon de Next.js pour un compte administrateur ou éditeur connecté ; les pages lisent alors la dernière version non publiée (`getDraft*BySlug`, `getDraftBusinesses` dans `src/lib/cms.ts`), avec `repli` sur la version publiée, affichent un bandeau et ne sont ni mises en cache ni indexées. Pas d'aperçu sur `home-page` et `about-page` (globals sans brouillons) ni sur `credentials` (sans page propre).

Localisation du **contenu** : `en` (par défaut), `fr`, `de`, avec `fallback: true`. Langue de l'**interface** d'administration : `fr`, `de` et `en` (`i18n.supportedLanguages`, `repli fr`), négociée depuis le navigateur et surchargeable par utilisateur. Tous les libellés du CMS (collections, groupes, champs, options, textes d'aide) passent par `src/payload/i18n.ts` — `tr(fr, de, en)` — de sorte que l'administration n'est jamais à moitié traduite. Toute nouvelle collection ou tout nouveau champ doit utiliser ce helper. Le sélecteur de langue de contenu (composant `localizer` de Payload) est masqué par CSS (`src/app/(payload)/custom.scss`) sur les écrans sans champ traduit — `compte`, `users`, `contact-submissions` — où il n'aurait aucun effet. L'avatar est l'avatar interne de Payload (`admin.avatar: 'default'`) : pas d'appel à Gravatar.

Collections

`expertise-areas` — Domaines d'expertise

`title`, `slug`, `summary`*, `icon`, `featuredOnHome`, `order`, `isPlaceholder`, onglet **Content** (`intro`, `challenges[]`, `services[{title, description}]`, `audiences[]`, `approach`), onglet **SEO**. Versions + brouillons. Alimente `/expertise` et `/expertise/[slug]`.

`experiences` — Expériences et projets

`title`, `slug`, `type_` (`assignment` | `project`), `organisation`, `role`, `sector`, `region_` (`europe` | `africa` | `international`), `countries[{code, name}]` (`code` ISO 3166-1 alpha-2 → carte et nom localisé ; `name` facultatif et localisé, prioritaire à l'affichage), `startDate`, `endDate`, `summary`, `featured`, `order`, `isPlaceholder`, onglet **Detail** (`context`, `responsibilities[]`, `resultsValidated`, `results[]`), onglet **Relations** (`expertiseAreas`, `relatedInsights`), **SEO**.

`results[]` n'est visible dans l'admin **et** sur le site que si `resultsValidated` est coché : aucun résultat non vérifié ne peut être publié par inadvertance.

`subscribers` — Abonnés newsletter

`email_` (unique), `locale_` (`fr` | `de` | `en`), `status_` (`pending` | `confirmed` | `unsubscribed`), `consentAt`, `confirmedAt`, `unsubscribedAt`, `source`. Création uniquement par l'API publique ; aucun jeton stocké (liens signés HMAC avec `PAYLOAD_SECRET`). Purge : `pending` > 7 jours, `unsubscribed` > 30 jours.

insights — Articles RK Insights

`title`, `slug`, `excerpt`, `coverImage`, `category`, `author`, `publishedAt`, `featured`, `readingTime` (calculé), `isPlaceholder`, onglet **Article** (`content` *), onglet **Relations** (`relatedExpertise`, `relatedInsights`, `relatedBooks`), **SEO**. Brouillons + **programmation de publication** (`schedulePublish`) + aperçu (voir « Aperçu des brouillons »). Le site n'affiche que les articles publiés dont `publishedAt` est passé. Newsletter : `sendNewsletter` (case à cocher), `newsletterSentAt` et `newsletterRecipients` (lecture seule) — l'article est envoyé une seule fois aux abonnés confirmés quand il est publié et que sa date est atteinte.

categories — Catégories éditoriales

`title`, `slug`, `description`. Neuf catégories créées par le seed.

books — Livres et publications

`title`, `slug`, `subtitle`, `author`, `cover`, `summary`, `description`, `audience[]`, `bookLanguage[]`, `format[]`, `isbn`, `publisher`, `publicationDate`, `pages`, `price`, `currency`, `stock` (vente directe, vide = non suivi), `availability`, `saleType`, `purchaseLinks[]`, `previewPdf`, `relatedBooks`, `featured`, `order`, `isPlaceholder`, `seo`.

`saleType` implémente le modèle hybride du cahier des charges : `external` (lien revendeur), `direct` (**modélisé mais non actif** — aucun paiement n'est simulé), `none` (information seule).

orders — Commandes (vente directe)

`number` (RK-AAAA-0001, unique), `status` (`pending` | `paid` | `shipped` | `cancelled` | `refunded`), `trackingUrl`, `provider` (`stripe` | `paypal`), `providerRef`, `locale`, `customerName`, `customerEmail`, `shipping{...}`, `items[{book | product, title, quantity, unitPrice, lineTotal}]`, `total`, `vatRate`, `vatAmount`, `currency`, `paidAt`, `shippedAt`, `digitalWaiverAt` (accès immédiat aux contenus numériques accepté), `note`. Création par l'API de paiement uniquement ; passage à `paid` uniquement par webhook Stripe signé ou capture PayPal, une seule fois — les produits numériques sont alors ouverts dans l'espace membre de l'acheteur. Passer à `shipped` envoie l'e-mail d'expédition.

products — Produits numériques

`title`, `slug`, `type` (`ebook` | `course` | `resource`), `price` (TTC EUR), `available`, `summary`, `cover`, `languages[]`, `description`, `ebookPdf`, `ebookEpub`, `resourceFile`, `modules[{title, lessons[{title, durationMinutes, content, videoUrl, attachment}]}]`, `featured`, `order`, `isPlaceholder`, `seo`. Brouillons. Les fichiers et le contenu des leçons (texte, vidéo, pièce jointe) ne sont **jamais** renvoyés par l'API publique : seuls les acheteurs y accèdent, par l'espace membre, après contrôle de leur accès. Titres et durées des leçons = programme public.

protected-files — Fichiers protégés (upload)

`title`. PDF, EPUB, ZIP, DOCX, XLSX, PPTX, stockés dans `private/files` (hors du dossier public, volume Docker `private`). Lecture réservée à l'équipe ; téléchargement par les acheteurs via `/api/members/download` (limite de 50 téléchargements par accès).

members — Membres

`email` (unique), `name`, `locale`, `lastLoginAt`. Créés automatiquement au premier achat d'un produit numérique. Connexion sans mot de passe par lien e-mail à usage unique (15 minutes ; 72 heures pour le lien envoyé après l'achat), session de 30 jours dans un cookie signé `httpOnly`.

entitlements — Accès aux produits

`member`, `product`, `order`, `grantedAt`, `downloads`, `completedLessons` (progression d'une formation). Créés au paiement ; un administrateur peut en ajouter ou en retirer à la main.

campaigns — Pages de campagne

`title`, `slug`, `summary`, onglet **Contenu** : `layout` (blocs, au moins un), onglet **SEO**. Brouillons et publication programmée. Adresse `/{{langue}}/campaigns/{{slug}}`, hors menu, présente dans le sitemap (sauf `noindex`). Blocs :

Bloc	Champs
hero	<code>eyebrow</code> , <code>heading</code> (vide = titre de la page ; H1 en premier bloc), <code>lead</code> , <code>image</code> , <code>cta</code>
text	<code>heading</code> , <code>content</code> (texte riche), <code>image</code> , <code>imagePosition</code> (<code>left</code> / <code>right</code>)
features	<code>heading</code> , <code>intro</code> , <code>items[{{title, description}}]</code> (1 à 12)
video	<code>heading</code> , <code>videoUrl</code> (YouTube / Vimeo, chargée au clic), <code>poster</code>
books	<code>heading</code> , <code>books</code> (livres publiés)
products	<code>heading</code> , <code>products</code> (produits numériques publiés)
faq	<code>heading</code> , <code>items[{{question, answer}}]</code> → données structurées <code>FAQPage</code>
cta	<code>heading</code> , <code>body</code> , <code>primary</code> , <code>secondary</code> (bande sombre)
newsletter	<code>heading</code> , <code>body</code> ; masqué tant que l'envoi d'e-mails n'est pas configuré

Boutons (`cta`, `primary`, `secondary`) : `{label, href}`. `href` = page du site commençant par `/` (le préfixe de langue est ajouté automatiquement) ou adresse `https://` (nouvel onglet) ; toute autre valeur est refusée à l'enregistrement (`src/lib/campaign-link.ts`). Clics mesurés par l'événement `campaign_cta_click`.

businesses — Écosystème entrepreneurial

`name`, `slug`, `tagline`, `description`*, `valueProposition`, `field`, `audience`, `website`, `contactEmail`, `logo`, `active`, `order`, `isPlaceholder`, `seo`. `active: false` masque l'activité sans la supprimer.

engagements — Conférences & médias

`title`, `slug`, `type` (`conference` | `workshop` | `panel` | `interview` | `podcast` | `video` | `press`), `date`, `endDate`, `summary`, `eventName`, `organiser`, `city`, `country` (ISO), `languages[]`, `cover`, `featured`, `isPlaceholder`, onglet **Description** (`description`), onglet **Vidéo et liens** (`videoUrl` YouTube/Vimeo validé, `externalUrl` https, `externalLabel`), **SEO**. Brouillons et publication programmée. Aucun contenu de démarrage.

Champs de classement : `topics` (relation multiple vers `expertise-areas`, seuls les domaines publiés sont affichés) et `durationMinutes`. **Médiathèque** `/media` : fiches passées dont le format est vidéo (`videoUrl` valide), `podcast`, `interview` ou `press` (`src/lib/media-library.ts`), filtrables par format, thème et langue. Une fiche avec vidéo et image publie un `VideoObject` (données structurées).

credentials — Formations et certifications

`title`, `institution`, `kind`* (`education` | `credential`), `year`, `location`, `description`, `order`, `isPlaceholder`. Affichées sur `/about`.

legal-pages — Pages légales

`title`, `slug`, `type` (unique : `imprint`, `privacy`, `cookies`, `terms`, `returns`), `needsLegalReview`, `content`, `lastUpdated`, `seo`. `needsLegalReview: true` ⇒ bandeau « brouillon » + `noindex` + exclusion du sitemap. Création/suppression réservées aux administrateurs.

media — Médiathèque (upload)

`alt` * (localisé, **obligatoire**), `credit`. Conversion WebP (qualité 82) et six déclinaisons : `thumbnail` 400×300, `card` 768×512, `portrait` 800×1000, `wide` 1600×900, `og` 1200×630, `book` 900 de large (proportions d'origine, les couvertures ne sont jamais recadrées). Types acceptés : JPEG, PNG, WebP, AVIF, SVG (`IMAGE_MIME_TYPES`). Limite 10 Mo.

Vidéos : aucune vidéo n'est téléversée ; elles sont ajoutées par lien YouTube ou Vimeo (`videoUrl` des conférences & médias, des blocs vidéo de campagne et des leçons). Un hook `beforeOperation` (`src/payload/upload-messages.ts`) refuse tout autre fichier **avant** le contrôle de Payload, avec un message dans la langue de l'administration (FR / DE / EN) : une vidéo renvoie vers YouTube / Vimeo et le champ *Vidéo*, tout autre fichier rappelle les formats acceptés (réponse 400, au lieu de « The following field is invalid: file »).

documents — Documents PDF (upload)

`title` _ (localisé), `kind` _ (`expert-profile` , `cv` , `book-extract` , `other`).

contact-submissions — Demandes de contact

`name` , `organisation` , `email` , `country` , `requestType` , `subject` , `message` , `locale` , `status` , `emailDelivered` , `consentAt` .

Création interdite via l'API publique : seule la route `/api/contact` écrit dans cette collection (API locale avec `overrideAccess`). Lecture réservée aux comptes authentifiés, suppression aux administrateurs.

Qualification (facultative) : `organisationType` , `budget` , `timeline` , `decisionRole` (listes fermées), `leadScore` (0–100, lecture seule) et `priority` (`high` | `medium` | `low` , modifiable). Grille de calcul et libellés : `src/lib/lead-score.ts` ; le score n'est jamais renvoyé au visiteur.

Export CSV (`subscribers` , `contact-submissions`) : boutons au-dessus de la liste

(`src/payload/components/ExportCsvButton.tsx` , `beforeListTable`) → `GET /api/admin/export/{collection}?`

`lang=fr|de|en[&status=confirmed]` . Réservé à un compte administrateur ou éditeur connecté (401 sinon), lecture avec ses droits (`overrideAccess: false`), 50 000 lignes au plus ; séparateur `;` , BOM UTF-8, protection contre l'injection de formules (`src/lib/csv.ts` , colonnes dans `src/lib/exports.ts`). L'export est journalisé (collection, nombre de lignes, identifiant de l'utilisateur), jamais son contenu.

Google Sheets (`subscribers` , `contact-submissions`) : hooks `afterChange` / `afterDelete` (`src/payload/hooks/sheets.ts`) → `src/lib/google-sheets.ts` (compte de service, JWT RS256, API Sheets v4, écriture `RAW` , file d'attente en mémoire, ligne retrouvée par l'ID en colonne A). Abonnés synchronisés seulement en `confirmed` / `unsubscribed` . Panneau `SheetsSync` au-dessus des listes ; `GET|POST /api/admin/sheets` (état / resynchronisation complète, compte de l'équipe connecté, origine vérifiée). Inactif sans `GOOGLE_SHEETS_SPREADSHEET_ID` , `GOOGLE_SERVICE_ACCOUNT_EMAIL` , `GOOGLE_SERVICE_ACCOUNT_PRIVATE_KEY` .

users — Comptes

`name` , `email` , `role` * (`admin` | `editor`). Session 8 h, verrouillage après 5 échecs pendant 10 minutes, cookies `SameSite=Lax` et `Secure` en production. Seul un administrateur modifie un rôle.

Globals

site-settings

- **Brand** : `name` , `headline` (localisé), `signature` , `logo` , `expertProfile` , `cvDocument` , `creditName` (crédit « © by ... » du pied de page, vide = masqué), `creditUrl` (lien facultatif)
- **Contact** : `email` , `phone` , `address` , `notificationEmail` , `spokenLanguages` , `bookingUrl` (https, outil de rendez-vous externe), `bookingLabel`
- **Social** : `social[{platform, url}]`
- **SEO defaults** : `defaultSeoTitle` , `defaultSeoDescription` , `defaultOgImage`

`appearance` — Apparence (Administrateur uniquement)

- **Couleurs** : `palette` (`signature` par défaut, `ivory`, `anthracite`, `petrol`, `forest`, `burgundy`, `custom`) ; si `custom` : `light`{`primary`, `accent`, `textSecondary`, `background`, `backgroundSubtle`} et `dark`{`background`, `backgroundSubtle`, `accent`} (hex, vide = valeur Signature)
- **Typographie** : `headingFont` (`source-serif`, `playfair`, `inter`)
- **Page d'accueil** : `hero`{`style`: `halo` | `plain` | `image`, `image`, `intensity`: `subtle` | `visible`}

Le moteur `src/lib/theme.ts` dérive de ces quelques couleurs l'ensemble des design tokens des deux thèmes et ajuste chaque couleur de texte pour atteindre le contraste WCAG AA ; la feuille générée est injectée dans le `<head>` (`#rk-appearance`). Palette Signature = aucune surcharge : les tokens réglés à la main dans `globals.css` s'appliquent. L'enregistrement déclenche la revalidation des pages (`revalidatePath`).

`shop-settings` — Réglages de la boutique

`enabled` (ouvre la vente directe ; sans effet tant que les clés de paiement manquent), `vatRate` (TVA incluse dans les prix, 0–30 %), `notificationEmail`.

`home-page`

`heroEyebrow`, `heroValueProposition`, `heroPortrait`, `heroKeyPoints[]`, `expertiseIntro`, `experienceIntro`, `ecosystemIntro`, `finalCtaTitle`, `finalCtaBody`, `seo`.

`about-page`

`lead`, `portrait`, `biography`, `career`, `vision`, `values[]`, `languages[]`, `regions[]`, `seo`.

Règles d'accès

Opération	Public	Editor	Admin
Lire un contenu publié	✓	✓	✓
Lire un brouillon	✗	✓	✓
Créer / modifier un contenu	✗	✓	✓
Créer / supprimer une page légale	✗	✗	✓
Modifier l'apparence du site	✗	✗	✓
Lire les demandes de contact	✗	✓	✓
Supprimer une demande de contact	✗	✗	✓
Voir / traiter les commandes	✗	✓	✓
Régler la boutique	✗	✗	✓
Voir les abonnés newsletter	✗	✓	✓
Modifier / supprimer un abonné	✗	✗	✓
Gérer les comptes et les rôles	✗	✗	✓

Implémentation : `src/payload/access.ts`.

Contenu de démarrage et repli

`src/content/starter.ts` contient le contenu livré avec le site. Il sert :

1. de source au script `npm run seed` (chaque entrée est marquée `isPlaceholder`) ;
2. de **repli** si `CMS_ENABLED=false` ou si la base est injoignable — le site reste debout, un avertissement est écrit dans les logs serveur et les fiches concernées affichent le bandeau « Contenu d'exemple ». Le repli n'est jamais silencieux.

Règles respectées dans ce fichier : aucun client, partenaire, diplôme, mission, résultat, témoignage ou statistique inventé ; tout élément attendu du commanditaire est écrit `[entre crochets]`.

Faire évoluer le modèle

1. Modifier les fichiers de `src/payload/`.
2. `npm run generate:types` (et `generate:importmap` si un composant d'admin personnalisé est ajouté).
3. Adapter les mappings de `src/lib/cms.ts` et les modèles de vue de `src/lib/types.ts`.
4. En production : créer et appliquer une migration (voir `DEPLOIEMENT.md` (Deploiement_et_exploitation.pdf) §4).